

GUÍA COMPLETA DE WIRESHARK

Para el examen de Seguridad de Datos

Filtros, análisis de tráfico, protocolos y detección de amenazas

Contenido basado en el examen ExamenEnero26.pcapng

APARTADO 1: Estadísticas y Tráfico General

a) Total de paquetes capturados

Para ver las estadísticas generales del archivo de captura:

Menú: Estadísticas → Propiedades del archivo de captura

En la tabla de estadísticas hay dos columnas clave:

- Capturado: número total de paquetes en el archivo
- Mostrado: paquetes visibles con el filtro actual

Si dice “Mostrado: 4712 (100.0%)” significa que hay 4712 paquetes en total y no hay ningún filtro aplicado.

Consejo

CONSEJO: Siempre mira la barra de estado inferior de Wireshark: muestra “Mostrado: X de Y” para contar paquetes rápidamente.

b) Porcentaje mostrado con filtro TCP

Cuando aplicas un filtro, la ventana de propiedades cambia:

`tcp`

Con este filtro aplicado: Capturado = 4712, Mostrado = 3649 (77.4%). Esto indica que 3649 paquetes de 4712 son TCP.

c) Identificar la IP del equipo que captura

Métodos para encontrar la IP local:

1. Mirar la columna Source de los primeros paquetes
2. Estadísticas → Endpoints → pestaña IPv4 (la IP con más tráfico suele ser la local)
3. Estadísticas → Conversaciones → IPv

Filtro para ver solo los paquetes emitidos por esa IP:

`ip.src == 192.168.0.`

d) Picos de tráfico — Gráfica E/S

Menú: Estadísticas → Gráficas de E/S (I/O Graphs)

Configuración recomendada:

- Intervalo: 1 segundo
- Añadir línea sin filtro para ver todo el tráfico
- Añadir línea con filtro `tcp.analysis.flags` para ver errores TCP
- Los picos indican ráfagas de tráfico en momentos concretos

APARTADO 2: Conexiones Remotas

a) Conexión FTP

a.a) Detectar si hay conexión FTP

Hay dos formas de filtrar el tráfico FTP:

Filtro por protocolo:

```
_ws.col.protocol == "FTP"
```

Filtro por puerto:

```
tcp.port == 21
```

Importante

IMPORTANTE: ftp.port NO funciona en Wireshark. Usa siempre tcp.port == 21 para filtrar por puerto FTP.

FTP usa el puerto 21 para control y el puerto 20 para datos. Para ver ambos:

```
tcp.port == 20 || tcp.port == 21
```

a.b) Paquetes FTP emitidos por nuestro equipo

Combina el filtro de IP origen con el de FTP:

```
ip.src == 192.168.0.200 && tcp.port == 21
```

O con protocolo:

```
ip.src == 192.168.0.200 && _ws.col.protocol == "FTP"
```

Consejo

CONSEJO: Mira la barra de estado inferior: "Mostrado: X de 4712" — X es tu respuesta.

a.c) Usuario FTP utilizado

FTP envía las credenciales EN TEXTO PLANO (sin cifrar). Para encontrar el usuario:

```
ftp.request.command == "USER"
```

En el panel de detalles verás: Request arg: nombre_de_usuario

Para encontrar la contraseña:

```
ftp.request.command == "PASS"
```

Importante

IMPORTANTE: FTP es un protocolo INSEGURO: usuario y contraseña viajan sin cifrar. Esto es un punto clave en seguridad.

a.d) Cierre de la conexión FTP

Para ver si se cierra la sesión:

```
ftp.request.command == "QUIT"
```

Para ver la respuesta de cierre del servidor (código 221):

```
ftp.response.code == 221
```

El mensaje típico es "221 Goodbye". El servidor es quien confirma el cierre.

Para ver todo el flujo de cierre:

```
ftp.request.command == "QUIT" || ftp.response.code == 221
```

b) Verificación DNS

b.a) IP del servidor DNS

Para encontrar el servidor DNS:

```
dns
```

La IP de destino de los paquetes de consulta (query) es el servidor DNS.

Filtro para ver solo consultas (sin respuestas):

```
dns.flags.response == 0
```

Consejo

CONSEJO: El servidor DNS suele ser el router local (192.168.x.1) o uno público como 8.8.8.8 (Google) o 1.1.1.1 (Cloudflare).

b.b) DNS hacia Google

Para verificar consultas DNS a dominios de Google:

```
dns.qry.name contains "google"
```

Esto muestra todas las consultas DNS que buscan resolver dominios con “google” en el nombre.

Otros filtros útiles:

```
dns.qry.name == "www.google.com"
```

```
dns.qry.name contains "google" && dns.flags.response == 0
```

APARTADO 3: Pings (ICMP)

a) Filtrar todos los pings

ICMP es el protocolo que usa el comando ping.

Filtro para ver request y reply:

```
icmp
```

Filtro solo para requests (tipo 8):

```
icmp.type == 8
```

Filtro solo para replies (tipo 0):

```
icmp.type == 0
```

Importante

IMPORTANTE: En Windows, cada ejecución del comando ping envía 4 paquetes. Si ves 16 requests → se ejecutó 4 veces ($16 \div 4 = 4$).

Para saber si todos reciben respuesta: compara el número de requests (tipo 8) con el número de replies (tipo 0). Si son iguales, todos respondieron.

b) Solo paquetes reply

```
icmp.type == 0
```

Mira la barra de estado para contar cuántos hay.

Otros filtros ICMP útiles:

Filtro Descripción

```
icmp.type == 3 Destination Unreachable  
icmp.type == 11 Time Exceeded (traceroute)  
icmp.code == 0 Código específico dentro del tipo
```

APARTADO 4: Acceso al Router (HTTP)

Detectar intentos de login al router

El router (192.168.0.1 o 192.168.1.1) se gestiona por HTTP. Las credenciales van en texto plano.

Filtros clave:

Filtro Descripción

```
ip.dst == 192.168.1.1 && http Todo el tráfico HTTP hacia el router  
ip.dst == 192.168.1.1 &&  
http.request.method == "GET"
```

Peticiones GET al router

```
ip.dst == 192.168.1.1 &&  
http.request.method == "POST"
```

Peticiones POST (formularios login)

http.authorization Cabeceras de autenticación HTTP Basic

Extraer usuario y contraseña

Método 1 — HTTP Basic Auth:

Busca la cabecera “Authorization: Basic ...”. El valor en Base64 contiene usuario:contraseña. Wireshark lo decodifica automáticamente en el panel de detalles: Credentials: usuario:contraseña.

Método 2 — Formulario POST:

Busca paquetes POST. En el cuerpo (body) verás campos como username=admin&password=1234.

Filtro combinado:

```
ip.dst == 192.168.1.1 && (http.authorization || http.request.method == "POST")
```

Consejo

CONSEJO: En tu examen se usó `http.request.method == "GET"` con destino 192.168.1.1. Busca en la URI patrones como `/cgi/cgi_authpage` para localizar los intentos de login.

Consejo

CONSEJO: Usa Follow TCP Stream (click derecho → Follow → TCP Stream) para ver la conversación HTTP completa en texto plano, incluyendo credenciales.

CHULETA DE FILTROS ESENCIALES

Filtros por protocolo

Filtro Descripción

```
tcp Todo el tráfico TCP  
udp Todo el tráfico UDP  
http Tráfico HTTP  
dns Consultas y respuestas DNS  
ftp Protocolo FTP (alias)  
icmp Pings (ICMP)
```

arp Resolución de direcciones
tls Tráfico cifrado TLS/SSL
ssh Conexiones SSH
telnet Conexiones Telnet
dhcp Asignación de IPs
smtp Correo saliente
_ws.col.protocol == "X" Filtrar por nombre exacto de protocolo

Filtros por IP

Filtro Descripción
ip.addr == 192.168.0.200 Origen o destino (cualquiera)
ip.src == 192.168.0.200 Solo como origen
ip.dst == 192.168.1.1 Solo como destino
ip.addr == 192.168.0.0/24 Toda la subred /
!(ip.addr == 192.168.0.200) Excluir una IP

Filtros por puerto

Filtro Descripción
tcp.port == 80 Puerto 80 (HTTP)
tcp.port == 443 Puerto 443 (HTTPS)
tcp.port == 21 Puerto 21 (FTP control)
tcp.port == 22 Puerto 22 (SSH)
tcp.port == 23 Puerto 23 (Telnet)
tcp.port == 53 Puerto 53 (DNS sobre TCP)
udp.port == 53 Puerto 53 (DNS sobre UDP)
tcp.srcport == 80 Puerto origen 80
tcp.dstport == 443 Puerto destino 443

Operadores lógicos

Filtro Descripción
&& o and Y lógico
|| o or O lógico
! o not Negación
== Igual a
!= Distinto de
> < >= <= Comparaciones numéricas
contains Contiene texto (subcadena)

Ejemplos combinados

Filtro Descripción
ip.src == 192.168.0.200 && tcp.port ==
21

FTP desde mi equipo

dns || icmp DNS o pings
!(arp) && ip.addr == 192.168.0.200 Mi tráfico sin ARP
http && !(ip.dst == 192.168.1.1) HTTP excepto al router
tcp.port == 80 || tcp.port == 443 HTTP o HTTPS
frame.len > 1000 Paquetes grandes (>1000 bytes)
tcp.analysis.flags Paquetes con errores TCP
http.response.code == 404 Respuestas "No encontrado"
http.response.code == 401 Login fallido (No autorizado)

MENÚS Y FUNCIONES CLAVE DE WIRESHARK

Menú Estadísticas (Statistics)

Filtro Descripción

Propiedades archivo captura Resumen general: paquetes, bytes, duración

Endpoints Lista de todas las IPs, MACs, puertos

Conversaciones Pares de comunicación (quién habla con quién)

Jerarquía de protocolos Desglose y % de cada protocolo

Gráficas E/S (I/O Graphs) Tráfico a lo largo del tiempo

Menú Analizar (Analyze)

Filtro Descripción

Follow TCP Stream Reconstruye conversación TCP completa

Expert Information Resumen de errores y avisos

Display Filters Gestionar filtros guardados

Importante

IMPORTANTE: Follow TCP Stream es tu mejor herramienta. Click derecho sobre cualquier paquete → Follow →

TCP Stream. Muestra conversaciones completas en texto plano (FTP, HTTP, Telnet...).

Colores en Wireshark

Filtro Descripción

Verde claro TCP

Azul claro UDP / DNS

Negro Errores TCP

Rojo RST (reset / conexión rechazada)

FILTROS AVANZADOS PARA RECUPERACIÓN

Análisis TCP avanzado

Filtro Descripción

`tcp.flags.syn == 1 && tcp.flags.ack == 0` Inicios de conexión (SYN)

`tcp.flags.syn == 1 && tcp.flags.ack == 1` Respuestas SYN-ACK

`tcp.flags.fin == 1` Cierres de conexión

`tcp.flags.reset == 1` Conexiones rechazadas/cortadas

`tcp.analysis.retransmission` Retransmisiones (problemas de red)

`tcp.analysis.duplicate_ack` ACKs duplicados (congestión)

`tcp.analysis.zero_window` Ventana TCP llena (receptor saturado)

HTTP detallado

Filtro Descripción

`http.request.method == "GET"` Peticiones GET

`http.request.method == "POST"` Peticiones POST

`http.request.uri contains "/login"` URIs con "/login"

`http.request.uri contains "/admin"` URIs con "/admin"

`http.host contains "google"` Peticiones a dominios Google

`http.response.code >= 400` Errores (cliente + servidor)

`http.response.code == 401` No autorizado (login fallido)

`http.response.code == 302` Redirecciones

`http.authorization` Cabeceras de autenticación

`http.content_type contains "html"` Contenido HTML

Detección de ataques y seguridad

Escaneo de puertos

Si una IP envía muchos SYN a puertos distintos sin completar el handshake = escaneo de puertos:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

ARP Spoofing

Muchas respuestas ARP sin petición previa son sospechosas:

```
arp.duplicate-address-detected  
arp.opcode == 2
```

Fuerza bruta

Muchos intentos de login seguidos al mismo servicio:

```
http.request.uri contains "login"  
ftp.request.command == "USER"
```

DNS Tunneling

Nombres DNS sospechosamente largos pueden indicar exfiltración de datos:

```
dns.qry.name.len > 50
```

Puertos sospechosos

Filtro	Descripción
tcp.port == 4444	Meterpreter (herramienta de pentesting)
tcp.port == 31337	Back Orifice (trojano clásico)
frame.len < 60	Paquetes anormalmente pequeños

TRUCOS CLAVE PARA EL EXAMEN

1. **Barra de estado (abajo):** Siempre muestra “Mostrado: X de Y”. Usa esto para responder a cualquier pregunta de “cuántos paquetes hay”.
2. **Follow TCP Stream:** Click derecho → Follow → TCP Stream. Ve toda la conversación FTP/HTTP de golpe. Ideal para ver usuario/contraseña.
3. **Columna Info:** Muestra resumen del paquete. En FTP verás “USER xxx”, “PASS xxx”, “QUIT”. En HTTP verás “GET /pagina HTTP/1.1”.
4. **Panel de detalles (centro):** Expande las capas: Ethernet → IP → TCP/UDP → Protocolo de aplicación.
5. **Endpoints:** Estadísticas → Endpoints te da TODAS las IPs del archivo. La que más tráfico tiene suele ser la local.
6. **Jerarquía de protocolos:** Estadísticas → Jerarquía de protocolos te dice qué protocolos hay y su porcentaje. Útil para descubrir rápidamente qué tipo de tráfico existe.
7. **Ping en Windows vs Linux:** Windows envía 4 pings por defecto. Linux envía pings hasta que lo detienes (Ctrl+C). Divide el total de requests entre 4 para saber cuántas veces se ejecutó en Windows.
8. **Protocolos inseguros (texto plano):** FTP, HTTP, Telnet envían credenciales sin cifrar. SSH, HTTPS, SFTP son sus versiones seguras.

Resumen: Protocolos y sus puertos

Filtro	Descripción
FTP (control)	Puerto 21 (TCP)
FTP (datos)	Puerto 20 (TCP)
SSH	Puerto 22 (TCP)
Telnet	Puerto 23 (TCP)
SMTP	Puerto 25 (TCP)
DNS	Puerto 53 (TCP/UDP)
HTTP	Puerto 80 (TCP)
HTTPS	Puerto 443 (TCP)
DHCP	Puertos 67/68 (UDP)

APARTADO 5: Análisis TLS 1.3 — Handshake en Wireshark

Basado en la Práctica 4 (captura de examen). IPs de ejemplo: cliente 192.168.0.198, servidor 192.168.0.196.

¿Puedes localizar ClientHello?

Sí. Suele aparecer en un paquete concreto (ej. paquete **308**): origen cliente → destino servidor. Columna **Protocol**: TLSv1.3.

¿Qué versión de TLS se está utilizando?

TLSv1.3 (columna Protocol en los paquetes del handshake).

¿Le sigue ServerHello?

Sí. Tras el ACK TCP del ClientHello, el servidor envía **Server Hello** (ej. paquete **310**).

¿Hay otros mensajes TLS integrados en el mismo paquete TCP que ServerHello?

Sí. En TLS 1.3 Wireshark agrupa en la columna **Info** del mismo paquete, por ejemplo: **Server Hello**, **Change Cipher Spec** y **Application Data** (datos ya cifrados).

¿Detectas luego el mensaje ClientKeyExchange?

Importante

IMPORTANTE (trampa de examen): En TLS 1.3 NO aparece un mensaje llamado ClientKeyExchange. El intercambio de claves va en los mensajes iniciales; lo siguiente suele etiquetarse como Application Data genérico (paquetes 310, 311, 312).

Justificación: el handshake TLS 1.3 es más corto que en TLS 1.2; Wireshark no nombra ClientKeyExchange porque ese mensaje no existe en 1.3.

¿Hay más mensajes del cliente integrados en el mismo paquete TCP?

Sí. En el paquete del cliente (ej. **311**) puede ir **Change Cipher Spec** + **Application Data** en el mismo segmento TCP.

¿Aparece finalmente el ChangeCipherSpec que envía el servidor?

Sí, pero **no al final** del handshake: aparece **adelantado**, integrado en el mismo paquete que el **Server Hello** (ej. paquete 310).

Filtros útiles para TLS en capturas:

`tls`

`tls.handshake.type == 1` → Client Hello

`tls.handshake.type == 2` → Server Hello

PREGUNTAS TIPO EXAMEN — Nmap y captura

Complemento de [ExamenNMAP_Auditoria.md](#) Bloque D. Ver también [Guia_Nmap.md](#).

- Durante `nmap -v -sn` verás ICMP Echo/Timestamp, TCP SYN (p. ej. 443) y TCP ACK (p. ej. 80); no hay escaneo de todos los puertos.
- Filtro solo tráfico hacia el objetivo → `ip.dst == 192.168.0.196`
- Tras el handshake TLS el HTTPS va cifrado; sin SSLKEYLOG no lees el contenido de aplicación.
- Con `nmap -sS` ves SYN y SYN-ACK, pero el escáner responde **RST** y no completa el three-way handshake (half-open).